CS224N Lecture 8: The Transformer Architecture (Part 2)

Stanford University

Spring 2023

Abstract

This lecture completes our study of the Transformer architecture by introducing the key components that make it work in practice: **multi-head attention**, **scaled dot-product attention**, **residual connections**, and **layer normalization**. We examine the complete Transformer decoder, encoder, and encoder-decoder architectures, understanding when to use each variant. Finally, we discuss the revolutionary impact of Transformers on NLP and their current limitations.

Contents

1 Introduction: From Minimal to Complete Architecture

1.1 The Gap

In Part 1, we introduced a **minimal self-attention architecture** with three components:

1. Position encodings
2. Self-attention mechanism
3. Feed-forward networks
4. Masking (for decoders)

However, this minimal architecture doesn't work as well as it could in practice. The complete **Transformer** adds several crucial components:

Key Additions in Transformer

1. **Multi-head attention**: Look at different aspects simultaneously
2. **Scaled dot-product**: Prevent gradient problems with large dimensions
3. **Residual connections**: Enable training of very deep networks
4. **Layer normalization**: Stabilize training dynamics

Remark 1.1. While the Transformer has been dominant since 2017, it's not necessarily the endpoint of architecture search. Understanding *why* each component is needed helps us think about potential improvements.

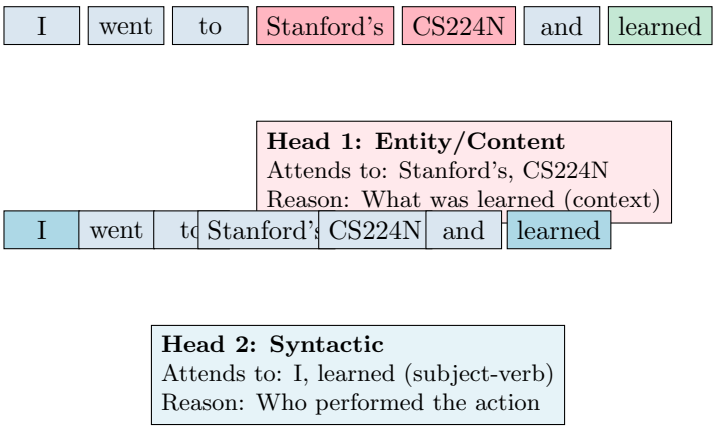
2 Multi-Head Attention

2.1 Motivation: Multiple Ways to Attend

Problem 2.1 (Single Attention Head Limitation)**.** A single attention mechanism must balance multiple competing objectives when deciding where to attend. Different types of relationships require attending to different parts of the sequence.

Example 2.1 (Multiple Reasons to Attend)**.** Consider: "I went to Stanford's CS224N and learned"

When representing "learned", we might want to attend for different reasons:



Problem: Single-head attention must average these different patterns, potentially losing important signal.

2.2 Multi-Head Attention: The Solution

Definition 2.1 (Multi-Head Attention)**.** Instead of computing attention once, compute h **independent** attention operations (heads) in parallel, each with separate learned parameters. Then combine the results.

2.2.1 Architecture

Multi-Head Attention Parameters

For h attention heads, define for each head $\ell \in \{1, \ldots, h\}$:

$$W_Q^{(\ell)} \in \mathbb{R}^{d \times (d/h)} \quad \text{Query projection} \tag{1}$$

$$W_K^{(\ell)} \in \mathbb{R}^{d \times (d/h)} \quad \text{Key projection} \tag{2}$$

$$W_V^{(\ell)} \in \mathbb{R}^{d \times (d/h)} \quad \text{Value projection} \tag{3}$$

Each head operates in a **lower-dimensional space** (d/h) rather than full dimensionality d.

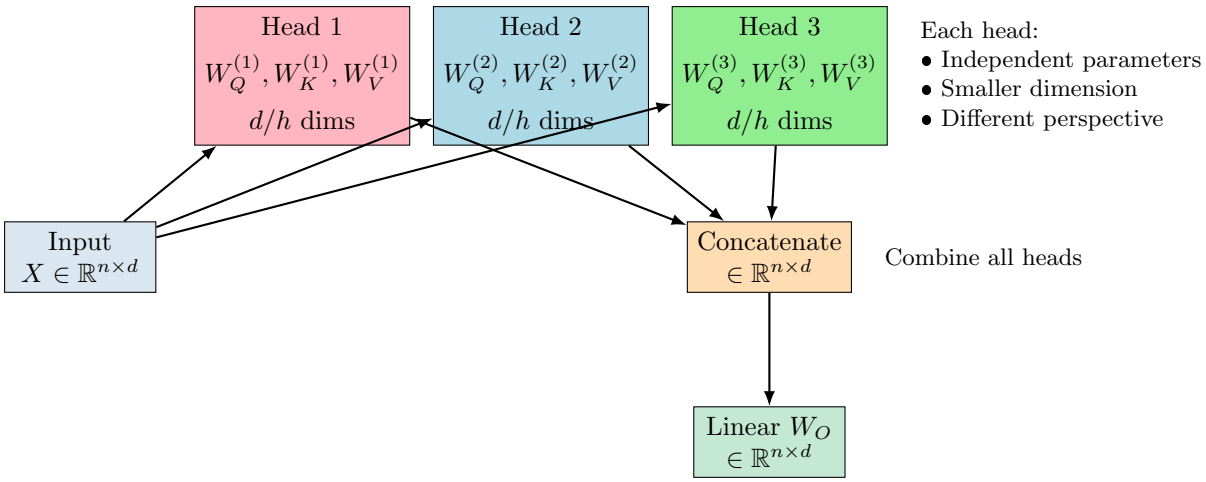
Output projection:

$$W_O \in \mathbb{R}^{d \times d} \tag{4}$$

2.2.2 Computation

Algorithm 1 Multi-Head Attention

Input: $X \in \mathbb{R}^{n \times d}$ (sequence of embeddings)
for $\ell = 1$ to h **do**
  $Q^{(\ell)} = XW_Q^{(\ell)}$ ▷ Project to queries
  $K^{(\ell)} = XW_K^{(\ell)}$ ▷ Project to keys
  $V^{(\ell)} = XW_V^{(\ell)}$ ▷ Project to values
  $\text{head}^{(\ell)} = \text{Attention}(Q^{(\ell)}, K^{(\ell)}, V^{(\ell)})$ ▷ Compute attention
end for
$\text{Concat} = [\text{head}^{(1)}; \text{head}^{(2)}; \ldots; \text{head}^{(h)}]$ ▷ Concatenate heads
$\text{Output} = \text{Concat} \cdot W_O$ ▷ Final linear projection
return Output



2.3 Efficient Implementation via Reshaping

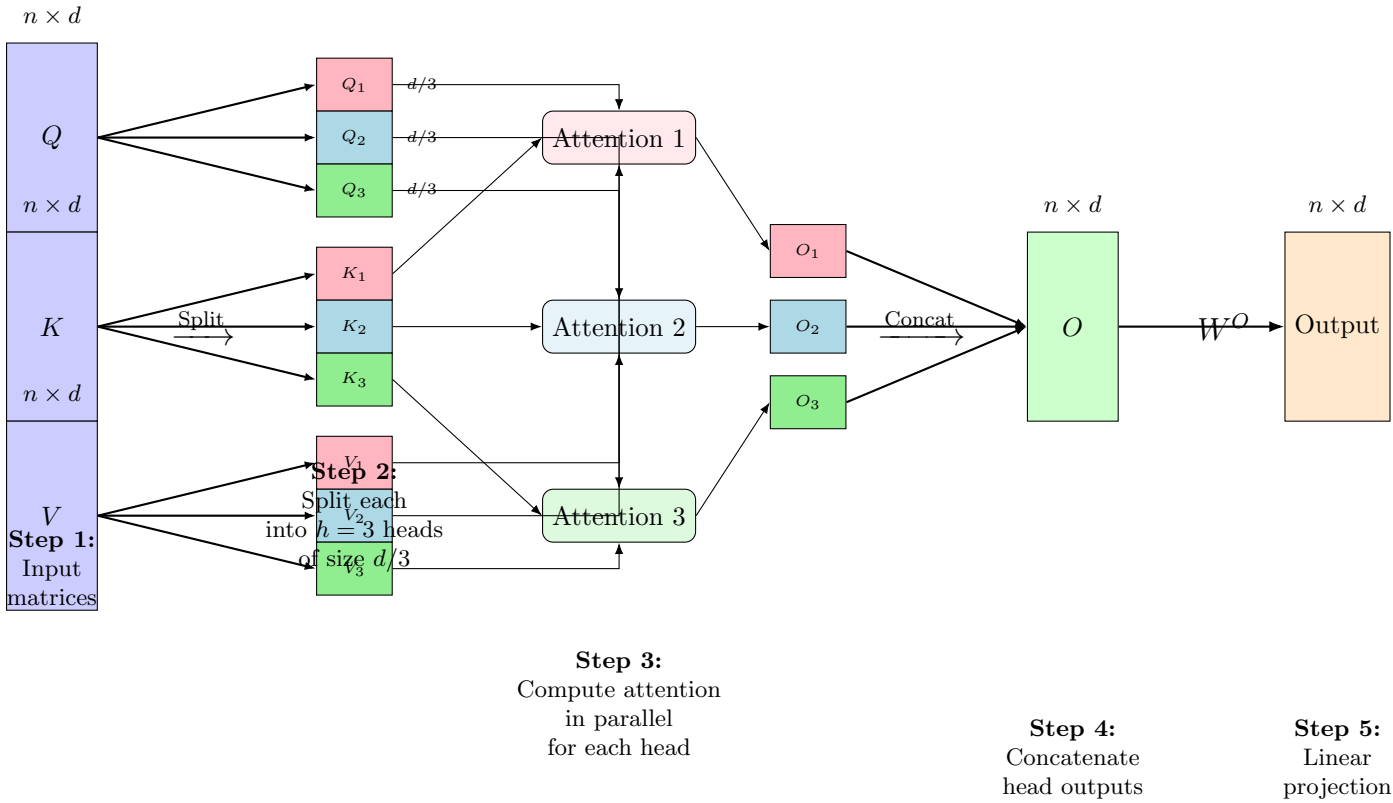
Key Insight: Multi-head attention costs about the same as single-head attention through clever tensor operations!

Tensor Reshaping Trick

Instead of: Computing h separate attention operations sequentially
Do: Reshape tensors to treat heads as a batch dimension

1. Compute $Q = XW_Q$ where $W_Q \in \mathbb{R}^{d \times d}$
2. **Reshape:** $Q \in \mathbb{R}^{n \times d} \to \mathbb{R}^{n \times h \times (d/h)}$
3. **Transpose:** $\mathbb{R}^{n \times h \times (d/h)} \to \mathbb{R}^{h \times n \times (d/h)}$
4. Now process all h heads in parallel (batch dimension)

2.3.1 Visual Example: 3-Head Attention



Key Points:

- All 3 heads run **in parallel** (no sequential dependency)
- Each head attends to different representation subspaces
- Total computation: same as single full-dimension attention
- Each head learns specialized attention patterns

2.4 Why Does Multi-Head Attention Work?

1. **Specialized attention patterns:** Different heads learn to focus on different aspects
   - Syntactic relationships (subject-verb, modifier-noun)
   - Semantic relationships (entities, coreference)
   - Positional patterns (previous word, next word)
2. **Ensemble effect:** Multiple independent views provide robustness
3. **Low-rank constraint:** Each head operating in d/h dimensions acts as regularization
4. **No additional cost:** Total parameters same as single head with full dimension

Remark 2.1 (Redundancy). Not all heads are equally important. Research shows:

- Some heads can be removed without hurting performance
- Heads do show some specialization but also redundancy
- The model hopes heads will specialize through training (not guaranteed)

2.5 Hyperparameters

Model	d **(model dim)**	h **(heads)**	d/h **(per head)**
Small	256	4	64
Base	512	8	64
Large	1024	16	64
GPT-3	12288	96	128

Table 1: Typical multi-head attention configurations

Rule of thumb: Keep $d/h \geq 64$ to give each head enough capacity.

3 Scaled Dot-Product Attention

3.1 The Problem with Large Dimensions

Problem 3.1 (Softmax Saturation)**.** When model dimensionality d is large, dot products between random vectors grow large in magnitude, pushing softmax into saturation regions with tiny gradients.

3.1.1 Mathematical Analysis

For two random unit vectors $q, k \in \mathbb{R}^d$ with i.i.d. components $\sim \mathcal{N}(0, 1/d)$:

$$\mathbb{E}[q^T k] = 0, \quad \text{Var}(q^T k) = 1 \tag{5}$$

But as d grows, the **standard deviation** of the dot product is $\sqrt{d}$, so values can be $\pm 3\sqrt{d}$ with high probability.

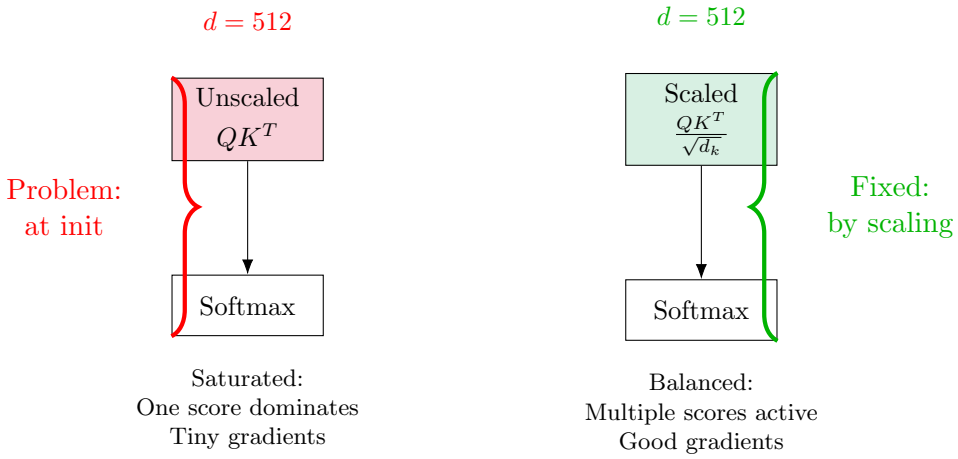
Example 3.1 (Softmax Saturation)**.** Consider $d = 512$:

- $\sqrt{d} \approx 22.6$
- Scores might be: $[45, 2, -30, 5, -20, \ldots]$
- After softmax: $[1.0, 0, 0, 0, 0, \ldots]$ (saturated!)
- Gradient: ≈ 0 for all but the largest score

3.2 Solution: Scaling

Definition 3.1 (Scaled Dot-Product Attention)**.** Scale attention scores by $\sqrt{d_k}$ where $d_k = d/h$ (dimension per head):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{6}$$



Why This Works

Scaling by $\sqrt{d_k}$ normalizes the variance:

- Before scaling: $\text{Var}(q^T k) \approx d_k$
- After scaling: $\text{Var}(q^T k/\sqrt{d_k}) \approx 1$
- Keeps scores in reasonable range at initialization
- Softmax doesn't saturate
- Gradients can flow

Remark 3.1. This is a simple fix (just one division) but crucial for training. Without it, Transformers train much more slowly or fail to converge.

4 Residual Connections

4.1 The Deep Network Problem

Problem 4.1 (Training Very Deep Networks)**.** As networks get deeper (10+, 50+, 100+ layers), they become:

- Harder to train (vanishing/exploding gradients)
- Prone to degradation (deeper $\neq$ better)
- Difficult to optimize

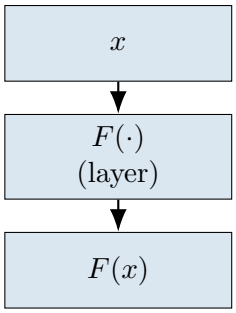
4.2 Residual Connections: The Solution

Definition 4.1 (Residual Connection)**.** Instead of learning $F(x)$ directly, learn the **residual** $F(x) - x$ and add it to the input:

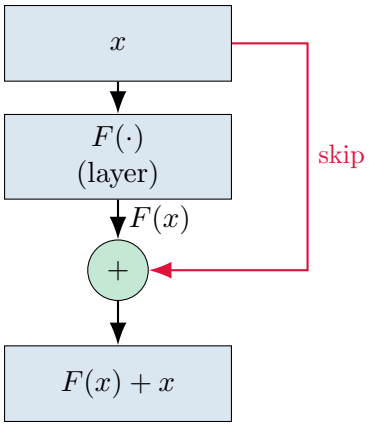
$$\text{Output} = F(x) + x \tag{7}$$

where F is any layer (self-attention, feed-forward, etc.)

Without Residual:



With Residual:



4.3 Why Residual Connections Work

1. **Gradient flow:**

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial (F(x) + x)} \cdot \frac{\partial (F(x) + x)}{\partial x} \tag{8}$$

$$= \frac{\partial \mathcal{L}}{\partial (F(x) + x)} \cdot \left(\frac{\partial F(x)}{\partial x} + 1 \right) \tag{9}$$

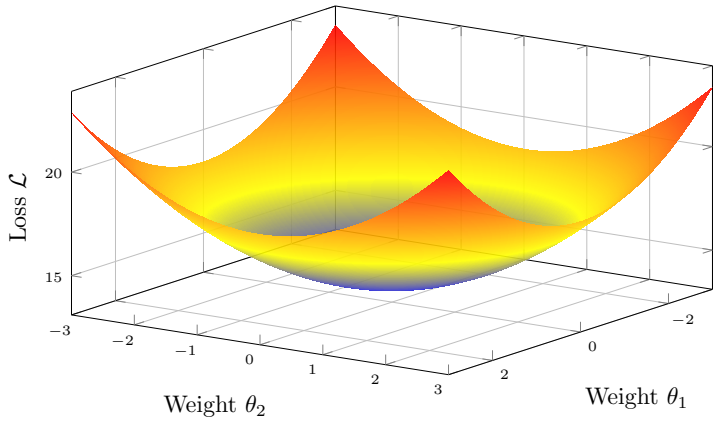
   The "+1" provides a **gradient highway**—even if $\frac{\partial F(x)}{\partial x}$ vanishes, gradient still flows through!

2. **Identity initialization:** At initialization, if $F(x) \approx 0$, then $F(x) + x \approx x$ (identity function). Network starts from a reasonable place.

3. **Easier optimization landscape:** Learning residuals $(F(x) - x)$ is often easier than learning full transformations.

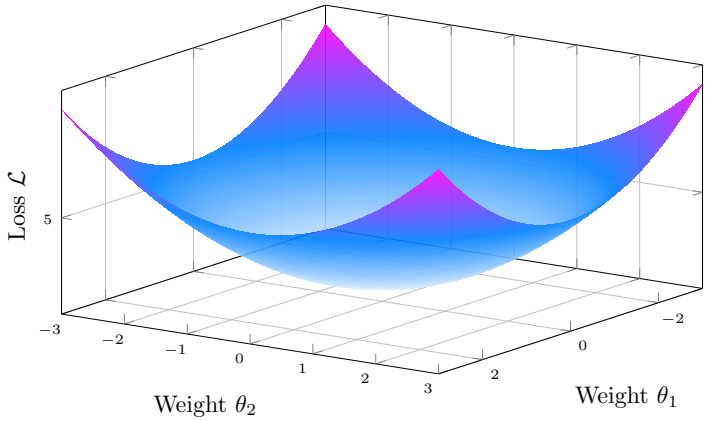
4.3.1 Loss Landscape Visualization

Without Residual Connections: Chaotic, Non-Convex



Problem: Many local minima, sharp valleys, and saddle points make gradient descent difficult and slow

With Residual Connections: Smooth, Nearly Convex



Benefit: Smooth bowl-shaped landscape enables faster, more reliable convergence to good minima

5 Layer Normalization

5.1 Motivation

Problem 5.1 (Training Instability)**.** During training, activations can have:

- Varying scales across layers
- Large or small magnitudes
- Distribution shifts

This makes optimization difficult and slows training.

5.2 Layer Normalization Definition

Definition 5.1 (Layer Normalization)**.** For a single vector $x \in \mathbb{R}^d$, normalize to zero mean and unit variance:

Compute statistics:

$$\mu = \frac{1}{d}\sum_{j=1}^{d} x_j \quad \text{(mean)} \tag{10}$$

$$\sigma^2 = \frac{1}{d}\sum_{j=1}^{d}(x_j - \mu)^2 \quad \text{(variance)} \tag{11}$$

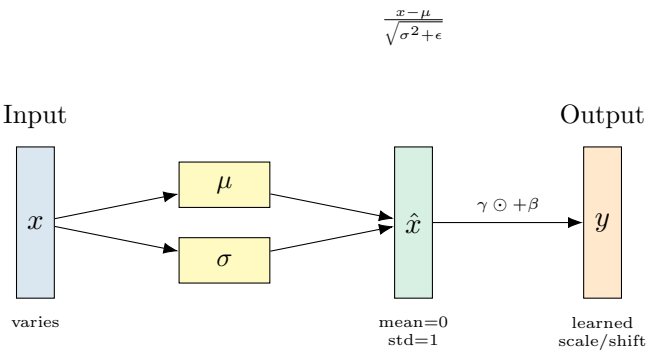
Normalize:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \tag{12}$$

Optional re-scaling and shifting:

$$\text{LayerNorm}(x) = \gamma \odot \hat{x} + \beta \tag{13}$$

where $\gamma, \beta \in \mathbb{R}^d$ are learned parameters, $\epsilon \approx 10^{-6}$ for numerical stability.



5.3 Key Details

Important Implementation Notes

1. **Normalize each vector independently:**
   - Don't share statistics across sequence positions
   - Don't share statistics across batch examples
   - Each position's statistics computed from its own features
2. **Normalize across feature dimension:**
   - μ and σ^2 computed over d dimensions
   - Not over sequence length n
   - Not over batch size B
3. **Re-scaling parameters γ, β are optional:**
   - Allow network to undo normalization if needed
   - In practice, often not critical

5.4 LayerNorm vs BatchNorm

Why LayerNorm for Transformers?

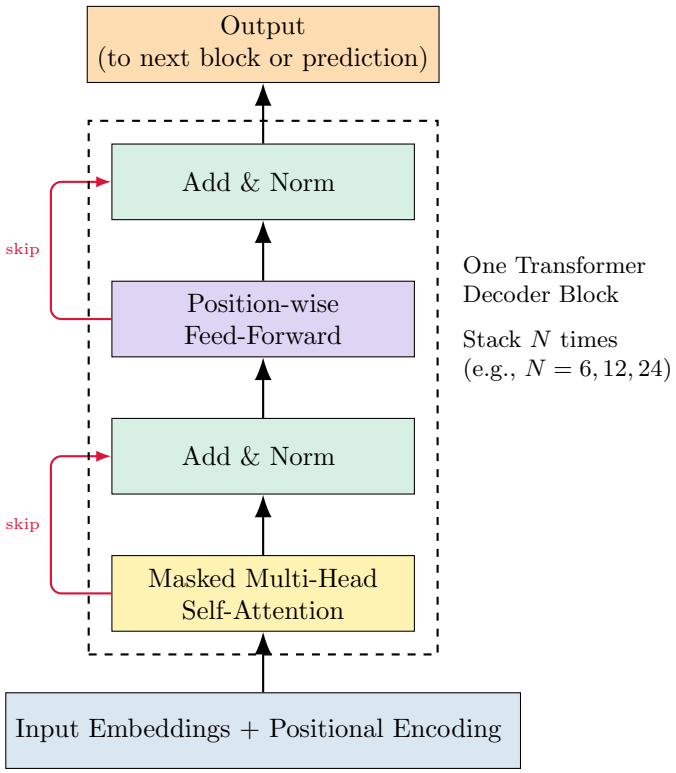
- Sequences have variable lengths
- Don't want coupling between examples in batch
- Same computation at training and inference

Property	Batch Norm	Layer Norm
Normalize over	Batch dimension	Feature dimension
Statistics shared across	Examples in batch	— (independent)
Depends on batch	Yes	No
Works for sequences	No (variable length)	Yes
Training vs inference	Different	Same

Table 2: Comparison of normalization methods

6 Complete Transformer Architecture

6.1 Transformer Decoder Block



6.1.1 Mathematical Formulation

Transformer Decoder Block

Input: $X \in \mathbb{R}^{n \times d}$
Block computation:

$$Z_1 = \text{LayerNorm}(X + \text{MultiHeadAttn}_{\text{masked}}(X, X, X)) \tag{14}$$
$$Z_2 = \text{LayerNorm}(Z_1 + \text{FFN}(Z_1)) \tag{15}$$

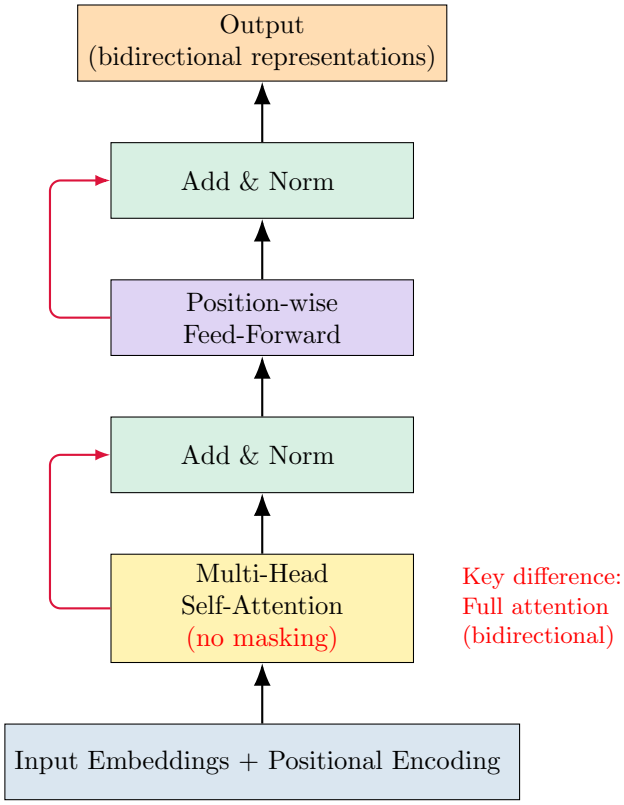
where:

- $\text{MultiHeadAttn}_{\text{masked}}$: Multi-head attention with future masking
- $\text{FFN}(z) = W_2 \cdot \text{ReLU}(W_1 z + b_1) + b_2$

Output: $Z_2 \in \mathbb{R}^{n \times d}$ (same shape as input, ready for next block)

6.2 Transformer Encoder Block

The encoder is nearly identical, but **without masking**:



Encoder vs Decoder

Encoder:

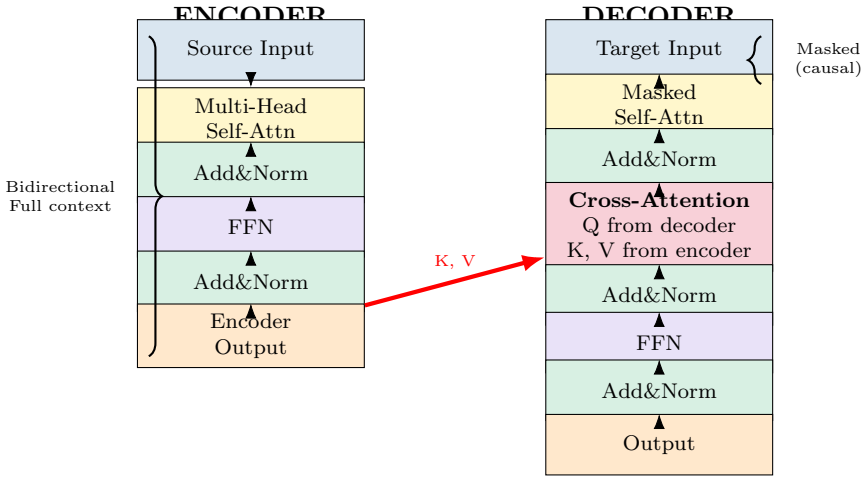
- **No masking** in self-attention
- Each position can attend to all positions
- Used for: BERT, document encoding, etc.

Decoder:

- **Masking** in self-attention
- Position i can only attend to positions $\leq i$
- Used for: GPT, language generation, etc.

6.3 Encoder-Decoder Architecture with Cross-Attention

For tasks like machine translation, we need both encoder and decoder plus **cross-attention**:



6.3.1 Cross-Attention Mechanism

Definition 6.1 (Cross-Attention)**.** Attention where queries come from one sequence and keys/-values from another:

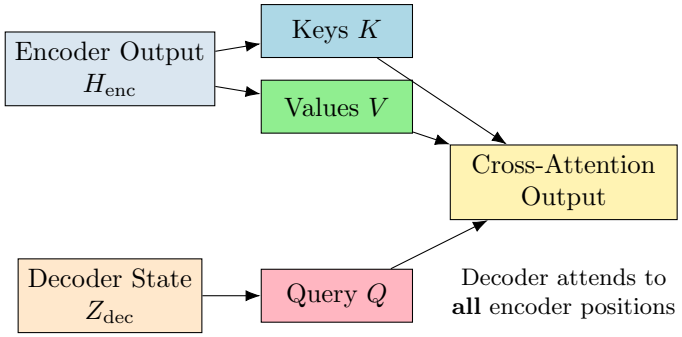
$$Q = Z_{\text{decoder}} W_Q \quad \text{(queries from decoder)} \tag{16}$$

$$K = H_{\text{encoder}} W_K \quad \text{(keys from encoder)} \tag{17}$$

$$V = H_{\text{encoder}} W_V \quad \text{(values from encoder)} \tag{18}$$

Then compute attention as usual:

$$\text{CrossAttn}(Z, H) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \tag{19}$$



Use cases:

- Machine translation (original Transformer)
- Summarization
- Question answering
- Image captioning (visual encoder, text decoder)

7 Transformer Variants Summary

Encoder-Only	**Decoder-Only**	**Encoder-Decoder**
Full self-attention	Masked self-attention	Both + Cross-attn
Bidirectional	Causal	
BERT, RoBERTa	GPT, GPT-2/3/4	T5, BART
Text classification	Text generation	Translation
Token tagging	Language modeling	Summarization

Architecture	Masking	Cross-Attn	Use Case
Encoder only	No	No	BERT, classification
Decoder only	Yes	No	GPT, language generation
Encoder-Decoder	Both	Yes	Translation, T5

Table 3: Transformer architectural variants

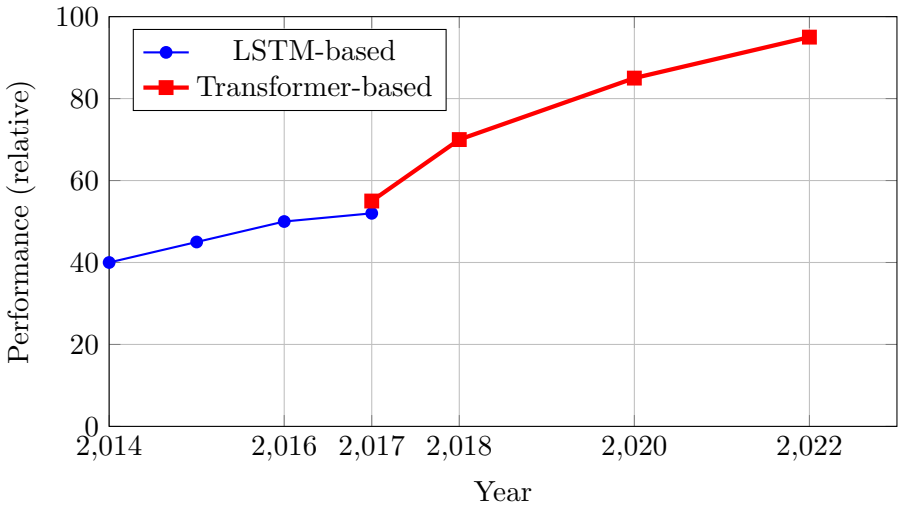
8 Impact and Results

8.1 Revolutionary Performance

Key Achievements

1. **Machine Translation (2017):**
   - Original "Attention Is All You Need" paper
   - SOTA results on WMT benchmarks
   - Crucially: Much faster to train than RNN models
2. **Scalability:**
   - Parallelization enables training on massive datasets
   - Can utilize modern GPUs effectively
   - Led to pre-training revolution
3. **Ubiquity (2018–present):**
   - BERT (2018): Pre-trained encoders
   - GPT-2/3 (2019/2020): Large-scale generation
   - Now: Virtually all SOTA models use Transformers

8.2 Performance Timeline



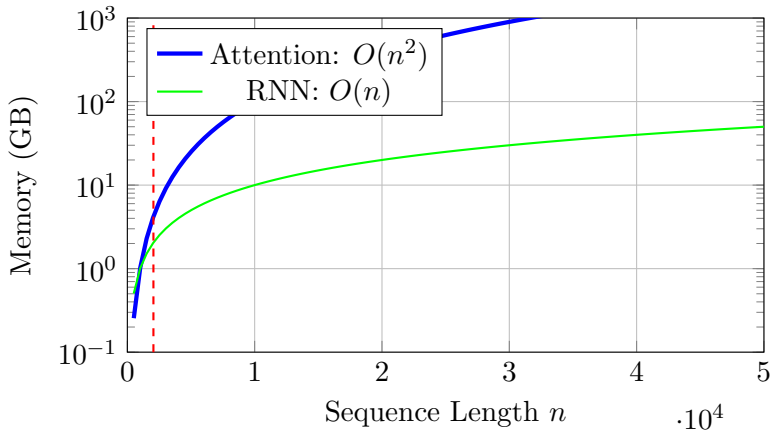
9 Limitations and Open Problems

9.1 Quadratic Complexity

Problem 9.1 (Memory Bottleneck)**.** Self-attention requires computing all pairs of attention scores:

$$\text{Memory} = O(n^2 \cdot d) \tag{20}$$

For long sequences, this becomes infeasible.



Example 9.1 (Practical Limits)**.** With $d = 1024$:

- $n = 512$: Memory $\approx$ 1GB (manageable)
- $n = 2048$: Memory $\approx$ 16GB (pushing limits)
- $n = 10000$: Memory $\approx$ 400GB (infeasible!)

9.2 Approaches to Address Quadratic Complexity

1. **Sparse Attention:**
   - Only compute attention for subset of pairs
   - Longformer, BigBird, etc.
2. **Low-Rank Approximations:**
   - Linformer: Project to lower dimension first
   - Reduces to $O(n)$ complexity
3. **Kernelized Attention:**
   - Reformer, Performer
   - Approximate attention with kernel tricks
4. **Recurrence:**
   - Transformer-XL: Cache previous segments
   - Trade memory for some sequential computation

Remark 9.1. Recent large models (GPT-3, GPT-4) find that **attention isn't the bottleneck** anymore—most computation is in the massive feed-forward networks. So quadratic attention may be acceptable!

9.3 Position Representation

Problem 9.2 (Absolute Position Issues)**.** Learned absolute position embeddings:

- Cannot extrapolate beyond training length
- May not capture relative relationships well
- Fixed maximum sequence length

Alternatives under research:

- Relative position encodings
- Rotary Position Embeddings (RoPE)
- ALiBi (Attention with Linear Biases)

9.4 Other Limitations

1. **Variable-length sequences:**
   - Padding to max length wastes computation
   - Need careful masking implementation
2. **Interpretability:**
   - What do different heads learn?
   - How does information flow through layers?
   - Active area of research
3. **Modifications landscape:**
   - Many proposed modifications
   - Few consistently improve over original
   - Original Transformer surprisingly robust

10 Summary

Key Takeaways

1. **Multi-Head Attention:**
   - Multiple independent attention operations
   - Each head can specialize (entities, syntax, etc.)
   - Implemented efficiently via reshaping
   - Crucial for Transformer performance
2. **Scaled Dot-Product:**
   - Divide scores by $\sqrt{d_k}$
   - Prevents softmax saturation in high dimensions
   - Simple but essential for training stability
3. **Residual Connections:**
   - Output = Layer$(x) + x$
   - Enables gradient flow in deep networks
   - Allows training 12, 24, 96+ layer models
4. **Layer Normalization:**
   - Normalize each vector to mean 0, std 1
   - Stabilizes training dynamics
   - Applied after each sub-layer
5. **Three Architectural Variants:**
   - **Encoder**: Bidirectional (BERT)
   - **Decoder**: Causal/masked (GPT)
   - **Encoder-Decoder**: Both + cross-attention (T5, translation)
6. **Impact:**
   - Revolutionized NLP (2017–present)
   - Enabled massive-scale pre-training
   - Now ubiquitous: BERT, GPT, T5, etc.
7. **Limitations:**
   - Quadratic memory in sequence length
   - Position encoding challenges
   - Active research on improvements

10.1 Looking Forward

Next lecture: Pre-training methods (BERT, GPT, masked language modeling)

Long-term: Transformers are dominant but not the end of the story. Understanding their components helps us think about future architectures.

11 Further Reading

11.1 Foundational Papers

1. **Attention Is All You Need:**
   Vaswani, A., et al. (2017). *NeurIPS.*
   The original Transformer paper.

2. **Layer Normalization:**
   Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer normalization. *arXiv.*

3. **Deep Residual Learning:**
   He, K., et al. (2016). Deep residual learning for image recognition. *CVPR.*

4. **BERT:**
   Devlin, J., et al. (2019). BERT: Pre-training of deep bidirectional transformers. *NAACL.*

5. **GPT-2:**
   Radford, A., et al. (2019). Language models are unsupervised multitask learners.

11.2 Efficient Transformers

- Longformer, BigBird (sparse attention)
- Linformer (low-rank approximation)
- Reformer, Performer (kernel methods)
- Survey: Tay et al. (2020), "Efficient Transformers: A Survey"

11.3 Visualizations and Tutorials

- The Illustrated Transformer: `https://jalammar.github.io/illustrated-transformer/`
- The Annotated Transformer: `http://nlp.seas.harvard.edu/annotated-transformer/`
- Transformer Explainer: `https://poloclub.github.io/transformer-explainer/`